



MÉMOIRE DE FIN DE 3^E ANNÉE · SECTION ALTERNANCE

D'une maquette à un **produit**.

Construire, professionnaliser et assumer le pilotage
d'une application de management des compétences

« Comprendre pour apprendre, décider pour faire grandir. »

AUTEUR

Maël GIRARDIN

ENTREPRISE D'ACCUEIL

A.F.D.E.C — Paris 20^e

PROJET

DevOPTIQ — application web SaaS

FORMATION

B.U.T. Informatique — 3^e année

MAÎTRE D'APPRENTISSAGE

Hubert GRANDJEAN

ANNÉE UNIVERSITAIRE

2025 – 2026

IUT — Bachelor Universitaire de Technologie, parcours Informatique.

Nota : ce mémoire reproduit, avec l'autorisation de l'A.F.D.E.C, des éléments de la méthode OPTIQ© et du code de l'application, faisant l'objet des dépôts : 00060838/2017-2019, 00079220-1/2022 et 000611383-1/2025.

Sommaire

— Remerciements

Celles et ceux qui ont rendu cette année possible

— Introduction

D'un projet d'étude à une responsabilité d'entreprise

I De l'environnement professionnel à l'évolution d'un projet

L'entreprise, la mission, et le basculement de la maquette vers le produit

- A. Présentation de l'A.F.D.E.C et de mon rôle
- B. Historique de l'entrée en apprentissage
- C. Le projet initial : la méthode OPTIQ et l'objectif « maquette »
- D. Le basculement : de la maquette au produit

II Développement — la traduction technique d'une ambition

Ce qui a été construit, pourquoi, et ce que les choix racontent

- A. Le développement d'une maquette poussée
- B. Le basculement dans le code — quatre franchissements
- C. La migration vers un produit : structure, tests, documentation

III Analyse critique, bénéfices et évolution

Regard lucide sur le travail, ses limites et ce qu'il a fait grandir

- A. Analyse des résultats obtenus
- B. Limites, biais et points d'évolution
- C. Bénéfices pour l'entreprise, pour le projet — et pour moi

— Conclusion

Ce que l'alternance laisse derrière elle

— Annexes

Glossaire, cartographie des modules, extraits de code, feuille de temps

Remerciements

es premiers remerciements vont à **Hubert Grandjean**, mon tuteur au sein de l'entreprise **M** qui m'a accueilli. Il y a plus d'un an, il m'a accordé sa confiance en me prenant en stage dans son entreprise et en me confiant le développement d'un projet innovant et ambitieux qu'il soutenait lui même depuis plusieurs mois. Cette année, il a fait bien plus que me renouveler sa confiance : il m'a associé progressivement, à la réflexion, aux décisions, et finalement à l'aventure elle-même. Je tiens aussi à le remercier pour son écoute attentive, ses conseils professionnels mais aussi plus personnels concernant l'avenir et pour sa bienveillance tout au long de mon alternance. En tant que maître d'apprentissage il m'a repris, corrigé et conseillé avec justesse et bienveillance. C'est grâce à ce dévouement et cette implication pour la cause du bon que j'ai pu tirer un tel enrichissement de cet alternance. Dans le monde professionnel, beaucoup se basent sur le savoir ou les performances brutes. Je crois que ce qui fait d'Hubert quelqu'un auquel j'apporte tout mon respect c'est qu'il croit avant tout en l'humain, il a compris que croire en quelqu'un et l'accompagner était la meilleure façon de le faire évoluer. Alors pour tout cela et cette expérience infiniment riche sur de multiples facettes, je le remercie.

Je remercie chaleureusement l'ensemble de l'équipe de l'**A.F.D.E.C** pour son accueil, sa disponibilité. Une mention particulière à **Véronique Decauwer**, dont l'aide sur tout ce qui touche à l'administratif — un terrain qui n'est pas mon point fort — m'a été très précieuse.

Un remerciement profond à Florent et Camille Girardin, qui m'ont mis en contact avec Hubert et qui m'ont permis de trouver ce stage cela n'aurait été possible sans eux. Ils m'ont aussi accueilli chez eux durant la période de stage et ont été d'une gentillesse infinie avec moi. Je ne compte les rires et les bons moments passés à leur côté, et je salue leurs attentions et leur intérêt à l'égard de mon stage et de ce que j'y entreprenais, ils ont été un soutien indéfectible et pour cela je les en remercie infiniment.

◆ — REMERCIEMENTS PERSONNELS

Je tiens évidemment également à remercier, ma famille; mon Papa et mes deux sœurs qui, malgré la distance, sont un soutien inébranlable pour moi.

D'un projet d'étude à une responsabilité d'entreprise

Ce mémoire retrace l'année d'alternance que j'ai effectuée au sein de l'A.F.D.E.C, en troisième année de B.U.T. Informatique. Il prolonge directement une histoire commencée un an plus tôt : celle d'un stage de deuxième année durant lequel j'avais repris le développement d'une application bâtie autour d'une méthode originale, la méthode **OPTIQ®**. Là où le stage m'avait permis de poser des fondations, l'alternance avait une ambition d'une autre nature : transformer ces fondations en un véritable produit.

Un mémoire d'alternance ne se contente pas de raconter ce qui a été fait. Il doit, au-delà du factuel, porter une **réflexion** sur les choix opérés, justifier la démarche, et faire apparaître ce que cette expérience a fait grandir en moi. C'est la ligne directrice que j'ai cherché à tenir tout au long de ces pages : ne jamais présenter une réalisation sans interroger le **pourquoi** qui la sous-tend, et ne jamais décrire une décision technique sans relier ce qu'elle dit de mon évolution — comme développeur, mais aussi comme collaborateur.

« Au départ, l'objectif était de développer la maquette d'une application future. Le projet a évolué, et notre objectif avec lui : il fallait désormais livrer un vrai produit, commercialisable. »

Cette phrase résume à elle seule le basculement qui structure tout ce mémoire. Pendant le stage, je construisais une **maquette** : une démonstration des idées, une preuve de concept destinée à convaincre. Pendant l'alternance, l'horizon s'est déplacé. Il ne s'agissait plus de montrer ce que l'application **pourrait** être, mais de la rendre solide, attrayante, fiable — bref, de la rendre **vendable**. Ce déplacement d'objectif a tout changé : la manière de structurer le code, le soin apporté au visuel, l'introduction d'une documentation et d'une batterie de tests, et jusqu'à la façon dont l'application sépare et protège les données de chaque client.

J'ai organisé ce mémoire en trois temps. La **première partie** situe l'environnement : l'entreprise, la méthode OPTIQ, mon rôle, et la manière dont le projet — et mes responsabilités — ont basculé d'une logique de maquette à une logique de produit. La **deuxième partie**, plus technique, entre dans le code : elle décrit l'ampleur de ce qui a été développé, puis met en regard quelques cas précis de « code d'hier » et de « code d'aujourd'hui » pour donner à voir, concrètement, ce que la professionnalisation a signifié. La **troisième partie**, enfin, porte un regard critique : elle analyse les résultats, assume les limites — notamment le choix de privilégier le fonctionnel et le visuel sur la sécurité dans un temps contraint — et mesure les bénéfices pour l'entreprise comme pour moi.

Une mention, dès l'introduction, est nécessaire. Comme lors de mon stage, l'**intelligence artificielle** a tenu une place importante dans ce travail — à la fois comme outil de développement et comme brique intégrée à l'application elle-même. Je l'aborderai sans détour, car elle fait pleinement partie de l'histoire de ce projet et de ce qu'il m'a appris.

De l'environnement professionnel à l'évolution d'un projet

Avant de parler de code, il faut parler de contexte. Une décision technique ne se comprend qu'à la lumière de l'entreprise qui la porte, de la méthode qu'elle défend, et de la trajectoire d'un projet qui, en un an, a changé de nature.

A. Présentation de l'A.F.D.E.C et de mon rôle

L'A.F.D.E.C est un laboratoire indépendant de recherche appliquée, spécialisé dans la modélisation des **organisations apprenantes et performantes**, dans le secteur privé comme public. Fondée sur des valeurs humaines fortes et une expertise pluridisciplinaire, l'association place la **co-construction** des solutions au cœur de sa démarche, afin de garantir leur durabilité et leur appropriation sur le terrain.

Concrètement, l'A.F.D.E.C intervient dans le management des compétences, l'amélioration des processus, la conduite du changement, le développement RH et l'évaluation des pratiques professionnelles. Dans le cadre de partenariats — en France comme à l'international — elle accompagne les organisations à chaque étape de leur évolution : de l'analyse des besoins à la mise en œuvre, en passant par la conception d'outils et de méthodes adaptés aux réalités du terrain. Elle est particulièrement reconnue pour ses solutions en **gestion prévisionnelle des emplois et des compétences (GPEC)**, en cartographie des compétences et en dispositifs d'évaluation des savoirs, savoir-faire et savoir-être.

REPÈRE Une recherche qui fait norme

Le travail de l'A.F.D.E.C alimente des **normes françaises** publiées — par exemple la **X50-124** (« piloter les compétences pour faire face aux transitions ») ou la **X50-766** (habiletés socio-cognitives). Ce détail n'est pas anodin pour mon travail : plusieurs modules de l'application s'appuient explicitement sur ces référentiels, ce qui imposait une fidélité méthodologique au-delà de la simple prouesse technique.

C'est une organisation qui mise avant tout sur la **confiance** accordée au collaborateur et sur le développement de ses compétences. Cette philosophie n'est pas un simple discours : je l'ai vécue de l'intérieur, et elle explique en grande partie la façon dont mon rôle a évolué.

Mon rôle, et son évolution

Je suis arrivé à l'A.F.D.E.C comme **stagiaire**, chargé de reprendre un développement en cours. Un an plus tard, j'en termine l'alternance dans une posture radicalement différente. Le glissement s'est fait par paliers, presque sans que je m'en aperçoive : d'abord exécutant de missions confiées, je suis progressivement devenu un interlocuteur avec qui l'on **réfléchit** le produit. Les échanges avec Hubert ont cessé d'être des transmissions de consignes pour devenir des discussions de conception — un rapport d'égal à égal, de construction commune, où mes initiatives pouvaient infléchir une idée de départ.



CE QUE L'ALTERNANCE A FAIT GRANDIR

La bascule la plus marquante n'est pas technique mais **relationnelle et décisionnelle**. Je ne « rendais » plus un travail : je co-portais une vision. Prendre des initiatives, défendre un choix d'architecture, accepter qu'une de mes idées soit rediscutée puis améliorée à deux — c'est cette posture de **co-décideur** que l'année m'a permis de travailler, bien plus que n'importe quelle ligne de code.

◆ À COMPLÉTER — FORMALISATION DE L'ASSOCIATION

Tu as évoqué être arrivé stagiaire et terminer « associé avec le PDG ». C'est un point fort de ton évolution, mais je ne connais pas les détails exacts (statut, date, périmètre). Préciser ici en une ou deux phrases la nature de cette association — c'est un excellent marqueur de la confiance accordée et de la montée en responsabilité.

B. Historique de l'entrée en apprentissage

Mon intégration en alternance s'est faite avec une aisance rare, et pour une raison simple : c'est dans cette même structure que j'avais réalisé mon **stage de fin de deuxième année**. Pendant douze semaines, j'avais déjà repris une application existante, analysé ses forces et ses faiblesses, et développé plusieurs modules. Quand le stage s'est terminé, le travail, lui, ne s'est pas arrêté : nous avons continué à collaborer durant l'été. L'application avait pris de l'ampleur, mais elle était loin d'être achevée.

L'alternance de troisième année est arrivée comme une évidence. Les trois mois d'un stage n'avaient jamais suffi à mener à terme un projet de cette envergure : il fallait du temps long, de la continuité, et la possibilité de penser le produit dans la durée plutôt que dans l'urgence d'une démonstration. L'alternance offrait précisément ce cadre — un fil continu sur une année entière, là où le stage n'avait été qu'une parenthèse intense.

CONTINUITÉ**Du stage à l'alternance**

Stage — 2 ^e année	Été (transition)	Alternance — 3 ^e année
12 semaines. Reprendre l'existant, consolider les bases, livrer une maquette à présenter à des investisseurs.	Poursuite de la collaboration. L'idée d'un produit complet commence à mûrir.	Une année. Transformer la maquette en produit : structure, visuel, documentation, tests, déploiement.

◆ À COMPLÉTER — DATES ET RYTHME EXACTS

Insérer les dates précises de l'alternance (début / fin), le rythme d'alternance IUT ↔ entreprise (nombre de semaines, présentiel / distanciel), et le nom du tuteur pédagogique. Je n'ai pas ces informations de façon certaine.

Le cadre de travail : agilité, autonomie et confiance

La méthode de travail s'est imposée naturellement, dans la continuité du stage : une approche **agile**, terrain familier pour Hubert comme pour moi — je l'avais découverte en deuxième année d'IUT. Concrètement, cela m'a laissé l'espace de coder, tester et proposer des solutions, puis d'en discuter pour ajuster, améliorer ou valider. Côté rythme, après une phase de présentiel pour bien m'imprégner de la vision du projet, le **distanciel** s'est installé comme une évidence : plus efficace, davantage d'autonomie, une organisation qui m'est propre, ponctuée de comptes rendus réguliers.

L'alternance a toutefois déplacé l'équilibre par rapport au stage. Là où j'étais d'abord un exécutant qui rendait des modules, je suis devenu un interlocuteur avec qui l'on pense le produit. L'autonomie s'est muée en **responsabilité partagée** : mes propositions pouvaient infléchir une orientation, et il m'a fallu apprendre à les défendre, à les nuancer, parfois à y renoncer au profit d'un compromis meilleur. Cette confiance — l'A.F.D.E.C en fait une valeur — a été le carburant le plus efficace de l'année.

C. Le projet initial : la méthode OPTIQ et l'objectif « maquette**»**

Pour comprendre le projet, il faut comprendre la méthode qu'il transpose. **OPTIQ®** est une approche d'analyse et de transformation des organisations. Elle repose sur une cartographie fine des activités et des compétences au sein d'une structure, dans le but d'en optimiser la performance globale. Elle permet de visualiser les connexions entre rôles, tâches et compétences clés, tout en révélant les zones de surcharge, de flou ou de sous-emploi. La méthode s'appuie sur deux outils complémentaires :

OUTIL 1 CartoColor®

Une représentation graphique et colorée des activités, à la manière d'une grande carte routière de l'organisation. Chacun y repère ses activités, se situe dans le flux d'informations et découvre son influence sur la performance collective. Le code couleur identifie immédiatement les compétences clés.

OUTIL 2 OPTIQFluent®

Le prolongement logique de CartoColor. Il travaille sur le traitement de l'information de chaque activité : identifier et faire progresser les compétences, définir les performances, analyser la charge de travail, passer des fiches de fonction aux **fiches de rôle**.

La richesse de la méthode tient à une progression. OPTIQFluent invite à passer des **fiches de fonction** — qui décrivent un poste — aux **fiches de rôle**, transversales et centrées sur les activités réellement exercées ; puis à définir des **statuts** qui clarifient les responsabilités et organisent l'autonomie de chacun. Toute la difficulté, pour le développeur, est de transposer cette finesse sans la trahir : une base de données trop rigide écrase la méthode, une interface trop verbeuse la rend illisible. Trouver ce point d'équilibre a été un fil rouge permanent.

L'A.F.D.E.C a souhaité transposer cette méthode dans une application, en plaçant la cartographie comme point d'entrée central. L'objectif affiché était simple à énoncer : **rendre la méthode OPTIQ accessible à tous**, donner à chaque collaborateur une vue claire sur ses compétences, ses rôles, les activités qui lui incombent et ses connexions au sein de l'organisation.

Mais, comme souvent, la simplicité de l'énoncé masquait la difficulté de la réalisation. Transposer fidèlement une méthode riche dans une application suppose un travail de **synthèse** permanent : tout représenter sans être ni verbeux ni redondant, et concevoir une base de données relationnelle qui reflète la méthode sans sacrifier la simplicité d'usage. À cela s'ajoute une contrainte que tout développeur connaît : une application qui transpose un objet existant sera **sans cesse** sujette à de nouvelles idées, à des changements de point de vue. Il fallait donc, dès le départ, une approche **modulaire et modulable**.

À ce stade — celui hérité du stage — l'ambition restait celle d'une **maquette** : une démonstration convaincante des idées de la méthode, suffisamment aboutie pour séduire des investisseurs, mais pas encore un produit que l'on installe chez un client, avec ses comptes, ses données cloisonnées et ses garanties de fiabilité.

D. Le basculement : de la maquette au produit

est ici que se joue le cœur de cette année, et la raison d'être de ce mémoire. Au fil des mois, un constat s'est imposé : la maquette était devenue trop convaincante pour rester une maquette. Ce qui n'était qu'une **preuve de concept** méritait de devenir un **produit**. Ce bascu-

lement n'a pas été décrété un matin : il a infusé dans nos discussions, puis il a reconfiguré tous mes objectifs de développement.

Passer de la maquette au produit, ce n'est pas « finir » l'application. C'est en changer la nature. Concrètement, ce virage s'est traduit par quatre chantiers que toute la suite de ce mémoire va décliner :

① Solidifier la structure

Robustesse de la base de données, migrations sûres en production, gestion d'erreurs défensive : une maquette plante en démo, un produit ne plante pas chez le client.

② Soigner le visuel

Une véritable identité visuelle — la charte rose / vert / blanc — pour passer d'un outil fonctionnel à un produit **désirable**, que l'on a envie d'utiliser.

③ Garantir la fiabilité

Une **documentation complète** et une **batterie de tests** couvrant l'application : un produit doit pouvoir rendre compte de son propre état de santé.

④ Penser le multi-client

Création de comptes, système d'**entités** pour cloisonner les données de chaque organisation : le marqueur le plus net du passage à une logique commerciale.



LE POURQUOI DE LA DÉCISION

Pourquoi basculer, alors qu'une maquette « suffisait » ? Parce qu'un produit fini **change la conversation**. On ne présente plus une idée à financer, on présente une solution à acheter. Ce déplacement engage l'entreprise vers une logique de commercialisation — et, pour moi, il a transformé un exercice de développement en un véritable projet d'**ingénierie produit**.

Ce basculement a aussi déplacé mes propres objectifs. Pendant le stage, ma réussite se mesurait à ce que je parvenais à **montrer**. En alternance, elle s'est mise à se mesurer à ce que l'application pouvait **encaisser** : un redémarrage serveur sans perte de données, un client qui ne voit jamais celles d'un autre, une fonctionnalité qui ne casse pas quand une clé d'API est absente. La partie suivante entre dans cette matière concrète.

Développement — la traduction technique d'une ambition

Une ambition ne vaut que par ce qu'elle devient dans le code. Cette partie décrit d'abord l'étendue de ce qui a été construit, puis met en regard quelques décisions techniques précises pour montrer, ligne à ligne, ce que « passer de la maquette au produit » a concrètement voulu dire.

A. Le développement d'une maquette poussée

Lorsque j'ai repris l'application, elle comptait une poignée de pages réellement utilisables. Aujourd'hui, DevOPTIQ est une application web complète, organisée en une trentaine de modules fonctionnels couvrant l'ensemble de la méthode OPTIQ — de la cartographie des activités jusqu'à la projection métier, en passant par l'évaluation des compétences, l'analyse du temps et la génération de plans de formation assistés par IA.

~55

BLUEPRINTS FLASK
(MODULES ROUTÉS)

29

MODÈLES DE
DONNÉES

63

GABARITS
(TEMPLATES)

~69k

LIGNES DE CODE
(PYTHON · JS · HTML)

Derrière chaque module se cachent des routes Python, des requêtes SQL, des interfaces construites et stylisées — mais surtout des idées et des échanges. L'architecture retenue est résolument **modulaire** : un blueprint Flask par domaine fonctionnel, des modèles SQLAlchemy centralisés, des gabarits Jinja2 qui s'assemblent par inclusion de partials, et un front en JavaScript **vanilla** (sans framework) pour garder la maîtrise totale du comportement des pages.

Un aperçu des grands domaines fonctionnels

Domaine	Ce qu'il apporte
Cartographie OptiqCarto	Éditeur SVG maison (formes, bandes, connexions, import de fichiers Visio <code>.vsdx</code>) et viewer. Point d'entrée central de la méthode.
Activités & tâches	Fiche et liste des activités, tâches, outils, contraintes, données d'entrée / sortie, liens inter-activités.
Compétences	Savoirs, savoir-faire, aptitudes, HSC ; évaluations multi-évaluateurs (Garant / Manager / RH), synthèse par rôle.
Gestion RH & comptes	Rôles, affectation des collaborateurs, managers, gestion des utilisateurs et des entités.
Performance & temps	Indicateurs de performance, analyse de la charge de travail par activité, rôle et utilisateur.
Briques d'IA	Propositions de compétences, plan de formation, onboarding, chatbot, import assisté, projection vers les fiches métier ROME.



VISUEL À INSÉRER — COMPARATIF « AVANT / APRÈS » DES PAGES DISPONIBLES

Reprendre l'idée du mémoire de 2^e année : une capture des quelques pages initiales en regard du menu actuel (la trentaine de modules). C'est la manière la plus parlante de matérialiser l'élargissement fonctionnel. (Captures d'écran à fournir.)

Trois modules emblématiques

Pour donner chair à cette ampleur, trois modules méritent un mot — chacun a représenté un défi particulier.

L'éditeur de cartographie OptiqCarto est, techniquement, la pièce la plus exigeante. C'est un éditeur SVG entièrement maison : l'utilisateur y dessine des formes, des bandes (les rôles), des connexions entre activités, et peut même importer un fichier Visio (`.vsdx`) existant. Chaque sauvegarde extrait les données du dessin pour les synchroniser avec les modèles de la base — la carte n'est pas une image, c'est une **structure de données vivante**. Les nombreuses itérations sur les ports de connexion, les badges de décision ou la persistance de l'état témoignent du raffinement que demande un éditeur graphique réellement utilisable.

La synthèse des compétences est sans doute la page la plus stratégique. Elle concentre la vision globale d'un collaborateur : ses rôles, ses activités, ses évaluations. La développer a

supposé de relier plusieurs couches de logique — récupération des données, affichage dynamique, édition restreinte selon les droits, écriture en base, historique. C'est là que j'ai le plus ressenti l'équilibre nécessaire entre structure **back-end**, interface et expérience métier ; c'est aussi la page qui a connu le plus de versions.

L'analyse du temps, enfin, donne de la profondeur aux données en croisant durée, récurrence, fréquence et délai pour en extraire des indicateurs de charge — par activité, par rôle ou par collaborateur. Elle m'a appris à concevoir une interface non comme un simple écran, mais comme un véritable **outil d'analyse** pour l'utilisateur final, capable de rester lisible malgré la richesse des informations qu'elle manipule.

Cette ampleur traduit une **priorité assumée**. Dans un temps contraint, nous avons d'abord chassé la richesse fonctionnelle — prouver tout ce que l'application pouvait faire — en laissant délibérément pour plus tard la sécurité de pointe, l'optimisation fine, les tests et la finition. Ce n'est pas un renoncement mais une **priorisation**, dont la Partie III mesurera les limites.

Mais l'ampleur n'est pas une fin en soi : elle est le décor sur lequel se détachent les décisions qui comptent vraiment. Plutôt que de survoler trente modules, j'en ai extrait **quatre cas** qui montrent, chacun à leur manière, le passage de la maquette au produit.

B. Le basculement dans le code — quatre franchissements

La Partie II.A a montré que l'application avait d'abord **grossi** — le réflexe de la maquette : prouver tout ce qu'on peut faire. Mais grossir n'est pas mûrir. Cette section montre l'autre mouvement, celui de la **nature** : l'instant où nos décisions ont cessé de viser la démonstration pour viser le produit. Pour le rendre tangible, chaque cas place côte à côte une version « maquette » et une version « produit » du même choix. On y retrouvera une constante : devenir un produit, c'est se soucier soudain de ce qu'une démo ignore — l'autonomie, l'isolation des données, la durabilité, le soin. Quatre **franchissements**, chacun avec sa décision, son **pourquoi**, et ce qu'il révèle de notre évolution.

► Cas n°1 — Notre propre outil : l'éditeur de cartographie **OptiqCarto**

LE PAS FRANCHI Cartographie dessinée sous Visio, un outil tiers → éditeur intégré, autonome, au cœur du produit

Voici le chantier le plus lourd de l'année, et sans doute le marqueur le plus spectaculaire du passage au produit. La cartographie — CartoColor — est le **cœur** de la méthode OPTIQ et le point d'entrée de l'application. Or, dans la maquette, cette cartographie n'existait pas **dans** l'application : elle était dessinée à l'extérieur, sous **Microsoft Visio**, puis exportée en fichier **.vsdx** que

l'application devait **parser** pour en extraire les activités et les liens. Toute la cartographie reposait donc sur un **outil tiers, propriétaire et payant**, et sur un parsing fragile du format XML de Visio.

Nous avons pris une décision forte : **construire notre propre éditeur de cartographie**, intégré à l'application. OptiqCarto est un éditeur SVG entièrement maison — l'utilisateur dessine ses activités, ses bandes (les rôles) et ses connexions directement dans le navigateur, sans aucun logiciel externe. À elle seule, cette brique représente plus de **6 500 lignes** de JavaScript. C'est le genre d'effort qu'une maquette ne justifie jamais, et qu'un produit exige.

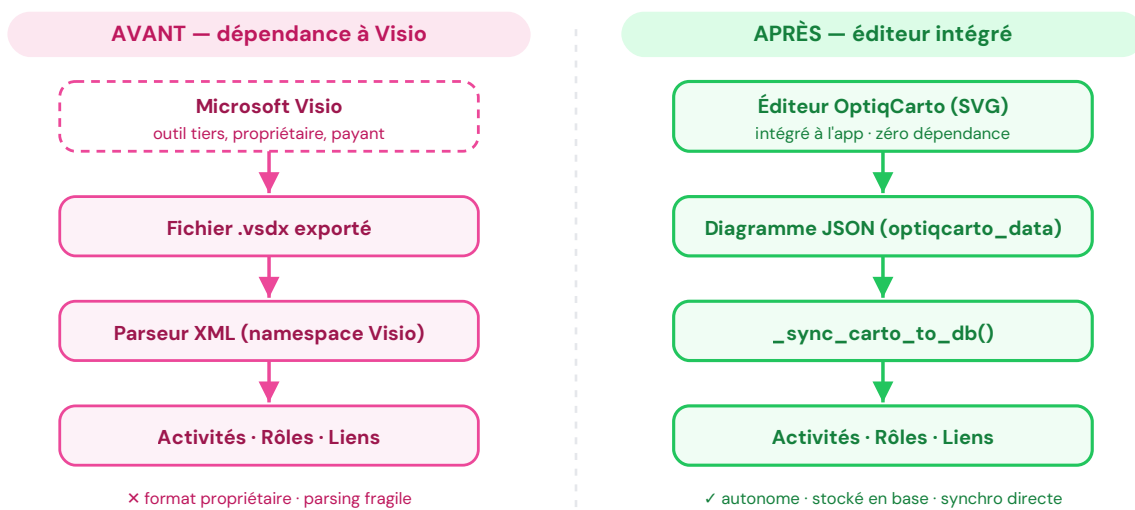


Figure 1. La cartographie quitte un outil externe (Visio) pour devenir une fonction native de l'application.

● AVANT — MAQUETTE · PARSEUR LE FORMAT PROPRIÉTAIRE DE VISIO

routes/vsdx_conexion_parser.py

```
class VsdxConectionParser:
    """Parse les connexions d'un fichier VSDX (format Microsoft Visio)."""
    VISIO_NS = {'v': 'http://schemas.microsoft.com/office/visio/2012/main'}
    EXCLUDED_LAYERS = {'6'} # hacks spécifiques au format Visio

    def parse(self):
        if not self.vsd_path.lower().endswith('.vsdx'):
            return [], ["Le fichier doit être au format .vsdx"]
        with zipfile.ZipFile(self.vsd_path, 'r') as zf:
            page_files = [f for f in zf.namelist() if 'pages/page' in f]
            # ... parcours fragile du XML propriétaire de Visio ...
```

```
@cartography_editor_bp.route("/api/save", methods=["POST"])
def api_save():
    entity = _get_active_entity()
    diagram = request.get_json(force=True)    # dessiné dans l'éditeur OptiqCarto

    # Le diagramme JSON est stocké en base → survit aux redémarrages cloud
    entity.optiqcarto_data = json.dumps(diagram, ensure_ascii=False)
    db.session.commit()

    # Re-extraction directe vers les modèles : activités, rôles, liens
    _sync_carto_to_db(entity, diagram)
    return jsonify({"ok": True})
```



CE QUE CE CHOIX TRADUIT

Internaliser la cartographie, c'est reprendre la main sur le **cœur** de la méthode. On supprime une dépendance à un logiciel tiers payant, on rend l'application **autosuffisante**, et on maîtrise tout le cycle : dessin → diagramme JSON → synchronisation directe en base. Un produit ne peut pas exiger de chaque client qu'il possède Visio. Construire notre propre éditeur — au prix de milliers de lignes et d'innombrables itérations (ports, bandes, connexions, sélection au lasso, liaisons inter-cartes) — était le **seul** chemin crédible vers un produit vendable.

Le détail technique est éloquent : le diagramme est sérialisé en JSON dans `optiqcarto_data`, donc il **survit aux redémarrages** du cloud, et chaque sauvegarde ré-extrait proprement les activités, rôles et liens vers la base sans détruire les données métier déjà rattachées. La cartographie n'est plus une image importée : c'est une **structure vivante**, née et gérée à l'intérieur du produit.

► Cas n°2 — Le virage commercial : le système **multi-entités**

LE PAS FRANCHI Les données d'une seule organisation → un espace cloisonné et étanche par client

Voici, selon moi, le marqueur le plus net du passage au produit. Notre maquette elle manipulait les données d'une organisation unique (celle visible). Notre produit commercialisable lui peut accueillir **plusieurs clients** et garantir, de façon stricte, qu'aucun ne voit les données d'un autre. Il a donc fallu introduire une notion d'**entité** (représentation d'une organisation, composé d'une carto et de sa structure de données attirée), un système de comptes, et cloisonner **chaque** requête par entité active.

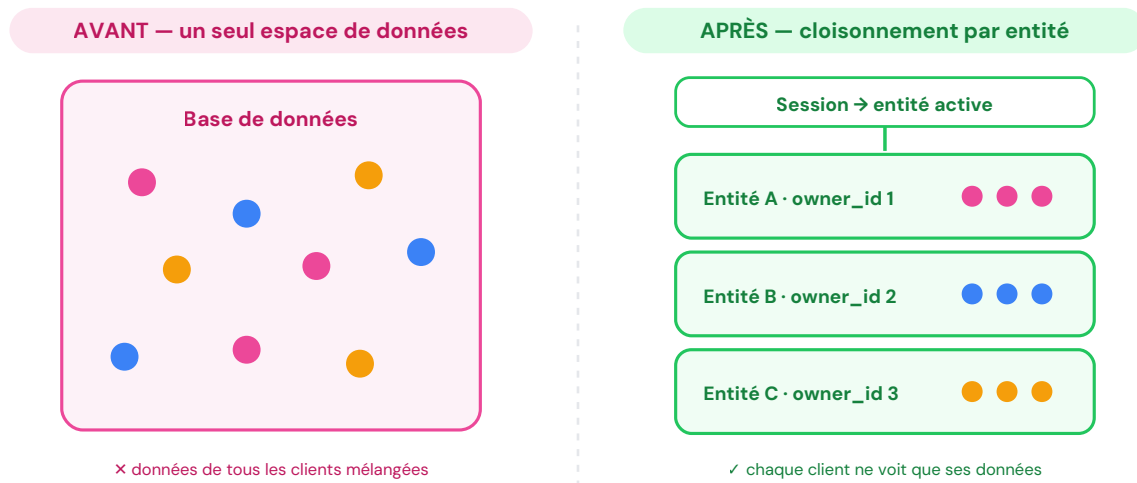


Figure 2. Le filtre `entity_id` / `owner_id` transforme un espace unique en autant d'espaces étanches qu'il y a de clients.

● **AVANT — MAQUETTE · UNE SEULE ORGANISATION, AUCUNE NOTION DE PROPRIÉTAIRE** `routes/activities.py` (représentatif)

```
@activities_bp.route("/activities")
def list_activities():
    # Toutes les activités de la base : une seule organisation implicite
    activities = Activities.query.all()
    return render_template("activities.html", activities=activities)
```



```
class Entity(db.Model):
    """Une organisation et sa cartographie. Chaque entité possède ses
    propres données et appartient à un utilisateur (owner_id)."""

    @classmethod
    def get_active(cls, user_id=None):
        if user_id is None:
            user_id = session.get('user_id')
        if not user_id:
            return None  # pas connecté → pas d'entité

        active_entity_id = session.get('active_entity_id')
        if active_entity_id:
            # STRICT : l'entité doit appartenir à l'utilisateur courant
            entity = cls.query.filter(
                cls.id == active_entity_id,
                cls.owner_id == user_id
            ).first()
            if entity:
                return entity
        # repli : la première entité de l'utilisateur
        return cls.query.filter_by(owner_id=user_id).first()
```

À partir de là, chaque route ne lit plus « toutes » les données, mais celles de l'entité active :

```
active = Entity.get_active()  # entité du client connecté
activities = Activities.query.filter_by(entity_id=active.id).all()
```



CE QUE CE CHOIX TRADUIT — ET SON AMPLEUR

Le cloisonnement par entité irrigue aujourd'hui l'ensemble du code — on compte plus de **340** filtres `entity_id` et **230** références à l'entité active à travers les modules. Décider cela, c'est accepter de repenser **chaque** requête de l'application. On passe donc d'une maquette modélisant une unique entité, à une application qui de par sa structure peut switcher entre diverses entités, ce qui en modifie sont contenu sans toucher à sa structure, ainsi même si la structure de l'application et l'organisation que ses données modélise sont détacher l'une de l'autre. Cela permet d'interchanger nos organisations modélisée (entités) au travers de notre infrastructure d'application. Cela fait basculer l'app dans une vraie vision modulaire du produit et des données qu'il contient.

Le commentaire même du modèle est révélateur de la maturité acquise : « **l'entité active est stockée dans la session utilisateur, pas en base** ». Ce genre de détail — où vit l'état, et pourquoi —

est typiquement ce à quoi on ne pense pas dans une maquette, et ce qu'on ne peut plus ignorer dans un produit multi-utilisateurs à vocation professionnelle.

► Cas n°3 — Penser le cloud : le **stockage des fichiers**

LE PAS FRANCHI Des fichiers écrits sur le disque, volatils en cloud → un stockage durable, en base

Ce cas est intéressant, car il illustre une vérité que la maquette ignore : **l'environnement de production a ses propres lois**. L'application est déployée sur Google Cloud Run, dans un conteneur dont le système de fichiers est **éphémère** : à chaque redémarrage ou redéploiement, tout ce qui a été écrit sur le disque disparaît. Une maquette qui enregistre les fichiers uploadés sur le disque local fonctionne parfaitement... jusqu'au premier redémarrage, où les fichiers des clients s'évanouissent.

● AVANT — MAQUETTE · ÉCRITURE SUR LE DISQUE LOCAL (VOLATIL EN CLOUD)

routes/upload.py
(représentatif)

```
UPLOAD_DIR = os.path.join(app.root_path, "uploads")

def upload_file():
    f = request.files["file"]
    path = os.path.join(UPLOAD_DIR, secure_filename(f.filename))
    f.save(path)                                # écrit sur le disque du conteneur
    return {"path": path}                       # ... perdu au prochain redémarrage
```

● APRÈS — PRODUIT · CONTENU BINAIRE PERSISTANT EN BASE (SURVIT AU CLOUD)

Code/models/models.py · routes/
export.py

```
class FileBlob(db.Model):
    """Contenu binaire des fichiers. Persistant en base
    (PostgreSQL BYTEA / SQLite BLOB) – survit aux redémarrages cloud."""
    filename = db.Column(db.String(255), nullable=False)
    mimetype = db.Column(db.String(128), nullable=False)
    data = db.Column(db.LargeBinary, nullable=False)

def upload_file():
    f = request.files.get("file")
    blob = FileBlob(filename=secure_filename(f.filename),
                    mimetype=f.mimetype, data=f.read())
    db.session.add(blob); db.session.commit()
    return {"path": f"/utils/file/{blob.id}"}    # servi depuis la base
```



CE QUE CE CHOIX TRADUIT

Stocker des fichiers en base de données n'est pas une « bonne pratique » universelle — c'est une **décision contextuelle**, justement raisonnée. Elle dit que j'ai cessé de coder « en général » pour coder **pour mon environnement réel**. La même logique a été appliquée au contenu des cartographies (SVG, données OptiqCarto), stockées en base plutôt que sur disque. Comprendre la cible de déploiement avant d'écrire le code : voilà un réflexe de produit, pas de maquette.

► Cas n°4 — Le soin du produit : **désirable** et **incroyable**

LE PAS FRANCHI Une démo qui peut être laide et fragile → un produit désirable, qui ne s'effondre jamais

Un produit se distingue d'une maquette notamment par deux exigences que celle-ci ignore : il doit donner **envie**, et il ne doit **jamais** s'effondrer (engendrer de bug ou d'erreurs non prévue) devant le client. L'esthétique et la robustesse semblent éloignées ; pourtant elles s'inscrivent dans la même logique : celle de faire d'une maquette un outil fini, durable et cohérent. Ainsi dans l'optique d'un produit complet, fini, ces deux finitions, longtemps reléguées au second plan dans la maquette, sont devenues des priorités du produit, lié au nouveau statut de l'application.

On va donc passer en revue 4 points qui représentent ce besoin de finition d'on nécessitait notre application.

Facette 1 — Une identité visuelle qui donne envie

Le style initial était fonctionnel mais trop brut : une feuille globale aux tailles énormes (**30px** partout), des couleurs nommées « en dur », et une cohérence globale pas voir peu présente lié a un développement axé sur le fonctionnel et étalé dans le temps.

En réponse a ce soucis, le produit s'est doté d'un véritable **design system** — des jetons de couleur (le rose **#ec4899**, le vert **#22c55e**, le blanc) utilisés dans plusieurs déclinaisons, sur toute l'application. Cela nous permet donc d'avoir une continuité et un cohérence visuelle entre toutes les sections que comporte l'application. Ce mémoire en est lui-même habillé : sa charte **est** celle de l'application.

● AVANT — MAQUETTE · COULEURS « EN DUR », PAS DE SYSTÈME

static/optiq.css

```
body { font-family: Arial; font-size: 30px; }      /* tout surdimensionné */

.update-btn {
  background-color: green;      /* couleur nommée, non maîtrisée */
  color: white;
}
```

```
:root {
  --pink: #ec4899; --pink-dk: #be185d; /* rose : couleur principale */
  --green: #22c55e; --green-dk: #15803d; /* vert : couleur d'accent */
}
.nav-logo { background: linear-gradient(135deg, var(--pink), var(--pink-dk)); }
```

Facette 2 — Une IA qui ne fait jamais planter le client

L'IA est fréquemment utilisée dans DevOPTIQ : elle propose des savoirs, des savoir-faire, des HSC, génère des plans de formation, alimente notre chatbot. Mais une dépendance externe est aussi une fragilité : et si la clé d'API est absente, le service indisponible, le quota épuisé ? Dans une maquette, on suppose que « ça marche ». Dans un produit, c'est interdit : une fonctionnalité ne doit **jamais** faire planter l'interface du client.

```
from openai import OpenAI
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"]) # KeyError si absente

def propose_savoirs(activity):
    resp = client.chat.completions.create(...) # 500 si l'API échoue
    return parse(resp)
```

```
def openai_client_or_none():
    """Instancie le client OpenAI ; si la clé manque → (None, raison)."""
    key = os.environ.get("OPENAI_API_KEY")
    if not key:
        return None, "Clé OpenAI manquante (OPENAI_API_KEY)."
    try:
        from openai import OpenAI
        return OpenAI(api_key=key), None
    except Exception as e:
        return None, str(e)

def dummy_from_context(ctx, kind="savoir"):
    """Repli déterministe : fabrique des items à partir du contexte.
    Ça évite de faire planter le front en prod."""
    ...
    return [f"{prefix} les procédures principales{titre_part}", ...]
```



CE QUE CES DEUX FINITIONS TRADUISENT

D'un côté, un langage visuel plus cohérent fait passer l'application d'une « c'est fonctionnel » à « on a envie de s'en servir ». De l'autre, le commentaire du code dit tout : « ça évite de faire planter le front en prod » — le client sans clé d'API obtient une proposition **dégradée** mais cohérente, et l'application reste debout. **Séduire et ne jamais décevoir** : ce sont les deux visages du même soin, celui d'un produit pensé pour son utilisateur final, et non pour une démonstration de ce que l'application peut faire.

SYNTHÈSE Ce que les quatre cas ont en commun

Pris ensemble, ces choix transforment la **nature** de l'application : la rendre autonome en internalisant son cœur — la cartographie (cas 1) ; multi-client et étanche (cas 2) ; durable sur le cloud (cas 3) ; enfin désirable et increvable (cas 4). Du plus lourd des chantiers au plus discret des replis, c'est ce travail souvent invisible qui sépare une maquette d'un produit — et c'est là que mon regard de développeur a le plus mûri cette année.

Ce dernier franchissement — le soin visuel — a quelque chose de particulier : ce n'est déjà plus une **fonctionnalité**, mais un **attribut de produit**, quelque chose qu'une démonstration n'a jamais à offrir. Et il en appelle d'autres, invisibles mais tout aussi décisifs : la fiabilité, la testabilité, la transmissibilité. Franchir ces étapes une à une ne suffisait pas ; il fallait, pour finir, **professionnaliser l'ensemble**. C'est l'objet de la dernière section.

C. La migration vers un produit : structure, tests, documentation

Si la partie B montrait des franchissements pris un à un, celle-ci montre un changement de régime. Trois chantiers transversaux ont accompagné le passage au produit : fiabiliser la base de données en production, se doter d'une **batterie de tests** qui rend compte de l'état de santé de l'application, et produire une **documentation** digne d'un logiciel que l'on transmet.

► Fiabiliser la base de données en production

La maquette tournait sur une base locale. Le produit tourne sur **PostgreSQL** dans le cloud, avec une base locale **SQLite** pour le développement. Concilier les deux, et faire évoluer le schéma sans jamais bloquer un déploiement, a demandé une stratégie de migration sur-mesure, exécutée au démarrage de l'application :

```
# Au démarrage : créer les tables manquantes, puis ajouter les colonnes
# manquantes une à une – idempotent, sans verrou DDL bloquant.
def _safe_add_column(table, col, col_type):
    try:
        with db.engine.connect() as _conn:
            _conn.execute(_text(f"ALTER TABLE {table} ADD COLUMN {col} {col_type}"))
            _conn.commit()
    except Exception:
        pass # colonne déjà présente – normal, on ignore

db.create_all() # tables manquantes
_safe_add_column("entities", "svg_content", "TEXT")
_safe_add_column("entities", "optiqcarto_data", "TEXT")
_safe_add_column("links", "choice_label", "VARCHAR(10)")
# ...
```



UNE DÉCISION NÉE D'UN INCIDENT DE PRODUCTION

Le code porte la cicatrice d'un vrai problème. À l'origine, les migrations passaient par l'outil standard (Alembic). En production, ce dernier **attendait un verrou PostgreSQL pendant plus de dix minutes** lorsqu'une colonne existait déjà — provoquant un **timeout** du worker, puis un crash en boucle. La décision, documentée dans le code, a été de **ne plus exécuter les migrations automatiques** et de les remplacer par des **ALTER TABLE** idempotents et instantanés. C'est un choix pragmatique — peut-être pas « académiquement pur » — mais qui a rendu les déploiements fiables. Apprendre à arbitrer entre la théorie et le terrain : voilà une compétence d'alternance.

Cette même recherche de robustesse se retrouve dans des dizaines de détails : des écritures en base conçues en « **supprimer puis réinsérer** » (UPSERT) pour résister aux particularités de PostgreSQL, ou encore un **journal d'activité automatique** qui, grâce aux **event listeners** de SQLAlchemy, enregistre seul chaque création ou modification — sans que les routes aient à y penser.

► Une batterie de tests qui rend compte de l'état de l'application

C'est peut-être le signe le plus visible de la professionnalisation. Là où la maquette n'avait aucun test, le produit en compte désormais une couverture étendue, organisée par module, exécutée sur une base SQLite temporaire dédiée aux tests :

979

TESTS
AUTOMATISÉS

41

FICHIERS DE TEST
(UN PAR DOMAINE)

~13k

LIGNES DE CODE
DE TEST

**/
testpanel**

PANNEAU DE TESTS
INTÉGRÉ À L'APP

Au-delà des chiffres, ces tests changent la façon de travailler : ils transforment la peur de « casser quelque chose » en confiance. On peut refondre un module en sachant qu'une régression sera signalée. Mieux encore, l'application embarque un **panneau de tests** (`/testpanel`) qui permet de visualiser, depuis l'interface, l'état de santé des différentes briques — une manière de rendre la fiabilité **lisible**, y compris pour un non-développeur comme Hubert.



VISUEL À INSÉRER — CAPTURE DU PANNEAU DE TESTS `/TESTPANEL`

Une capture du panneau de tests (résultats verts / rouges par module) illustrerait parfaitement cette idée de « produit qui rend compte de son propre état ». (Capture à fournir.)

► Documenter : transmettre plutôt que détenir

Un produit que l'on veut commercialiser ne peut pas reposer dans la seule tête de son développeur. J'ai donc rédigé deux documentations complémentaires, elles aussi habillées de la charte rose / vert / blanc : une **documentation technique** (architecture, modèles de données avec diagrammes, référence des API, conventions, déploiement) et un **guide utilisateur** destiné aux futurs clients.

POUR L'ÉQUIPE Documentation technique

Diagrammes d'architecture et de données, flux de démarrage, référence des modules et des endpoints, stratégie de déploiement. Le socle pour qu'un autre développeur puisse reprendre le projet.

POUR LE CLIENT Guide utilisateur

Une prise en main pas-à-pas des fonctionnalités, pensée pour l'utilisateur final. Le signe que l'on s'adresse désormais à des **clients**, pas seulement à des investisseurs.



CE QUE L'ALTERNANCE M'A APPRIS ICI

Tester et documenter sont, pour un étudiant, les tâches les plus faciles à négliger : elles ne « se voient » pas. C'est en entreprise, avec l'horizon d'un produit transmis et vendu, que j'ai compris leur valeur réelle. Écrire 979 tests et deux documentations n'a rien d'héroïque ; c'est simplement ce que **fait** un professionnel qui pense à l'après. Cette discipline est sans doute l'acquis le plus transférable de l'année.

Analyse critique, bénéfices et évolution

Un travail sérieux se juge aussi à la lucidité de son auteur sur ses propres limites. Cette partie assume les choix, expose les biais, et mesure ce que cette année a réellement apporté — à l'entreprise, au projet, et à moi.

A. Analyse des résultats obtenus

Le résultat le plus tangible est qu'il existe désormais un **produit**, là où il n'y avait qu'une maquette. Cette phrase paraît modeste ; elle résume pourtant un déplacement profond. L'application n'est plus une démonstration d'idées : c'est un logiciel multi-clients, déployé en production sur le cloud, doté d'une identité visuelle, d'une documentation et d'une couverture de tests, et capable d'accueillir l'organisation d'un client réel sans exposer ses données.

Sur le plan strictement fonctionnel, l'ambition de départ — transposer la méthode OPTIQ — est largement tenue : la cartographie, le cœur de la méthode, est éditable et exploitable ; les compétences, les rôles, les performances et le temps sont gérés ; et plusieurs briques d'IA viennent assister l'utilisateur là où la méthode est la plus exigeante. Mesuré à l'aune du mémoire de deuxième année, qui décrivait « huit modules » comme un aboutissement, l'élargissement est net.

BILAN

Ce qui est atteint, ce qui reste ouvert

Atteint	Encore ouvert
Produit déployé, multi-entités, cloisonnement strict des données.	Durcissement de la sécurité (voir B).
Identité visuelle cohérente sur toute l'application.	Optimisation fine des performances (requêtes, cache).
979 tests automatisés + documentation technique et utilisateur.	Stratégie de distribution et de commercialisation.

B. Limites, biais et points d'évolution

J'aborde ce point sans détour, car il est au cœur de la démarche réflexive attendue. Le choix le plus structurant de l'année a été un **arbitrage** assumé : dans un temps contraint, nous avons privilégié **ce qui se voit et ce qui sert** — le fonctionnel et le visuel — au détriment de la sécurité de pointe et de l'optimisation. Il faut en mesurer précisément la portée, sans le minimiser ni le dramatiser.

La sécurité : ce qui est protégé, ce qui ne l'est pas encore

Il serait faux de dire que la sécurité a été ignorée. La sécurité **qui compte le plus** pour un logiciel multi-clients — **l'isolation des données** entre organisations — est en place et stricte : chaque entité est rattachée à son propriétaire, et les requêtes refusent de servir les données d'autrui. Les mots de passe ne sont jamais stockés en clair ; ils sont hachés (**werkzeug**).

En revanche, la **couche de durcissement défensif** reste à construire. Pour être lucide :

- Le contrôle d'accès est fait « au cas par cas » (vérification de session dans les routes) plutôt que par un garde systématique (**décorateur**) appliqué uniformément.
- Il n'y a pas encore de protection **CSRF** sur les formulaires, ni de limitation de débit (**rate limiting**) contre les abus.
- Les en-têtes de sécurité HTTP et la politique de contenu ne sont pas durcis (le chiffrement TLS, lui, est assuré par Cloud Run en amont).



POURQUOI CE CHOIX — ET POURQUOI IL EST DÉFENDABLE

Face à un temps limité, il fallait d'abord **exister comme produit** : démontrable, désirable, vendable. Un produit magnifiquement sécurisé mais sans fonctionnalités ne convainc personne ; un produit fonctionnel et séduisant peut être **durci ensuite**, par couches. Ces manques ne sont donc pas des oublis mais une **dette technique consciente**, identifiée et priorisée — exactement le type d'arbitrage qu'impose la réalité de l'entreprise, et que la formation, centrée sur la rigueur individuelle, prépare peu.

L'optimisation, et la vraie limite : la distribution

Dans le même esprit, l'optimisation fine (élimination des requêtes redondantes, mise en cache) a été laissée pour plus tard : tant que les volumes restent ceux d'une jeune solution, la priorité était ailleurs. Mais la limite la plus importante n'est, paradoxalement, pas technique. Nous disposons aujourd'hui d'une application techniquement solide ; le défi suivant est celui de la **distribution** : comment la porter jusqu'à ses clients, l'installer, la facturer, l'accompagner. Le centre de gravité

du projet se déplace du développement vers le **go-to-market** — un terrain nouveau, et passionnant.

Une limite vécue : le relationnel à l'international

Une difficulté plus personnelle mérite d'être nommée. Le projet a une dimension internationale — des échanges, notamment, avec des interlocuteurs en Inde, en anglais. J'y ai éprouvé une limite réelle : le relationnel professionnel dans une langue qui n'est pas la mienne demande un effort que le code, lui, ne réclame pas. C'est un axe de progrès clairement identifié, et un rappel utile que le métier ne se réduit pas à la technique.

◆ À COMPLÉTER — ANECDOTE RELATIONNELLE / INTERNATIONALE

Si tu le souhaites, illustrer ce point par un exemple concret (une réunion en anglais, la présentation en Inde, un échange avec des développeurs ou investisseurs indiens). Un fait précis rendra cette limite plus vivante et sincère. Je n'ai pas les détails exacts pour l'écrire à ta place.

Regards croisés : ma formation à l'épreuve du produit

Une analyse critique serait incomplète sans confronter cette expérience à ma formation. Sur bien des points, l'IUT m'avait **bien préparé** : la pile technique (HTML, CSS, JavaScript, SQL) m'était familière depuis la deuxième année orientée développement web ; Python, moins évident au départ, s'est révélé proche des langages objet déjà maîtrisés ; la méthode agile et la culture du compte rendu, des rapports et des points réguliers venaient directement des projets collectifs de la formation. Ces **convergences** ont accéléré mon travail et m'ont évité de me former en urgence.

Mais l'alternance a aussi révélé des **divergences** instructives, là où l'entreprise diffère de l'école.



L'IA : OUTIL DE DÉVELOPPEMENT ET MATIÈRE DU PRODUIT

La formation aborde peu l'IA comme outil — c'est compréhensible dans une logique d'évaluation des compétences propres de l'étudiant. En entreprise, son usage est non seulement toléré mais recommandé : pour accélérer des tâches redondantes, comprendre vite une erreur complexe. J'ai appris à m'en saisir tout en gardant un **regard critique** et un rôle de décisionnaire : l'IA n'est pas une alternative au développement, c'est un outil de plus dans l'arsenal. Et, fait notable, elle n'est pas seulement **mon** outil : elle est **intégrée au produit lui-même**, où elle propose compétences, savoirs et HSC à partir de la structure d'une activité. Apprendre à la maîtriser des deux côtés — comme développeur et comme concepteur — est l'un des apprentissages les plus actuels de l'année.

Deux autres divergences m'ont marqué. La **responsabilité**, d'abord : le travail que l'on fournit n'est plus seulement pour soi ou pour une note, il engage le projet, l'équipe, l'entreprise — et le regard que l'on porte sur son propre code en est transformé. La **continuité**, ensuite : travailler chaque

jour, sur le même produit, pendant une année entière, est une expérience que les projets morcelés de la formation ne reproduisent pas. C'est précisément cette continuité qui rend possible le passage de la maquette au produit — un basculement qui demande du temps long, et une mentalité d'**ingénierie produit** que l'école, par nature, n'enseigne qu'en théorie.

C. Bénéfices pour l'entreprise, pour le projet — et pour moi

Pour l'entreprise et le projet

Le premier bénéfice est d'ordre stratégique. En faisant le choix d'un déploiement **conteneurisé (Docker) sur Cloud Run**, l'application reste légère à exploiter : l'A.F.D.E.C n'a pas à **sur-investir** dans des serveurs ou des services cloud coûteux. On paie l'usage, l'application est portable d'un environnement à l'autre, et la facture reste proportionnée à une jeune solution. Pour une association, cette frugalité d'infrastructure n'est pas un détail : c'est ce qui rend le projet soutenable.

Le deuxième bénéfice est patrimonial. L'A.F.D.E.C dispose désormais d'un **actif** — un produit documenté, testé, déployé — et non plus d'une simple démonstration dépendante de son auteur. La documentation et les tests garantissent que ce patrimoine est **transmissible** : il pourra être repris, audité, étendu. C'est précisément ce que l'on attend d'un logiciel destiné à être commercialisé.

BÉNÉFICE CLÉ Une donnée de test pensée pour le commercial

Un détail révélateur : la base de données embarque ses propres **données de test** (modèles d'utilisateur, d'entité...). Ce n'est pas anecdotique : cela traduit la réflexion derrière le virage commercial. Penser dès la conception à la création de comptes et au cloisonnement par entité, c'est concevoir l'application **comme** un produit que plusieurs clients utiliseront en parallèle — et non comme la vitrine d'une seule organisation.

Pour moi : ce que l'alternance a fait grandir

L'évolution personnelle est, au fond, le vrai sujet de ce mémoire. Elle s'est jouée sur plusieurs plans.

Une montée en responsabilité. Je suis arrivé stagiaire ; je termine bien plus engagé dans le projet, jusqu'à une forme d'association avec son porteur. Ce glissement n'est pas qu'un titre : c'est un changement de regard sur mon propre travail, qui n'est plus « rendu » mais **porté**.

Un passage de l'exécution à la co-construction. Le rapport avec Hubert a cessé d'être une transmission de missions pour devenir un échange de conception : je propose, il challenge, on arbitre. La prise d'initiative — déjà esquissée pendant le stage — est devenue le mode normal de

travail. J'ai appris à défendre un choix d'architecture, mais aussi à accepter qu'il soit transformé par la discussion.

Une maturité d'ingénieur produit. Techniquement, l'année m'a fait passer du « faire marcher » au « faire tenir » : penser le déploiement avant le code, anticiper les pannes, cloisonner les données, tester, documenter. Ce sont des réflexes que la formation aborde en théorie et que seule la confrontation à un vrai produit ancre durablement.



LA SYNTHÈSE DE L'ÉVOLUTION

De la **maquette** au **produit**, et du **stagiaire exécutant** au **co-décideur associé** : les deux trajectoires sont la même histoire. En apprenant à faire grandir un projet, j'ai grandi avec lui. C'est ce parallèle, plus que n'importe quelle ligne de code, que je retiens de cette année d'alternance.

Ce que l'alternance laisse derrière elle

A terme de cette année, il m'est difficile de séparer ce que j'ai construit de ce que je suis devenu. Les deux se sont façonnés ensemble. J'avais repris une maquette ; je laisse un produit. J'étais arrivé pour exécuter ; je repars en ayant co-décidé. Entre les deux, il y a eu des centaines d'heures de code, mais aussi — et c'est l'essentiel — un changement de regard sur ce que signifie « faire du logiciel ».

Ce mémoire a cherché à montrer que la technique n'est jamais neutre : derrière un système de jetons de couleur, il y a la volonté de séduire un client ; derrière un filtre `entity_id`, le respect des données d'autrui ; derrière un fichier stocké en base, la compréhension d'un environnement réel ; derrière un `fallback`, le refus de laisser tomber l'utilisateur. Chaque décision technique racontait une intention, et chaque intention disait quelque chose de la **maturité** que l'alternance m'a permis d'acquérir.

J'ai aussi appris à être lucide. À nommer ce qui n'est pas fini — la sécurité à durcir, l'optimisation à mener, la distribution à inventer — non pas comme des échecs, mais comme la carte du chemin qui reste. Un produit n'est jamais « terminé » ; il entre dans une nouvelle phase de sa vie. Et l'A.F.D.E.C aborde cette phase avec un actif solide, frugal à exploiter, documenté et testé : une base saine pour la suite.

« En apprenant à faire grandir un projet, j'ai grandi avec lui. »

Sur le plan humain, cette alternance a prolongé et approfondi ce que le stage avait ouvert : le goût des petites structures où chacun compte, la valeur d'une communication régulière, et la chance — rare — de travailler dans un climat de confiance et de reconnaissance mutuelle. Elle m'a aussi confronté à mes limites, notamment relationnelles à l'international, et m'a donné envie de les travailler.

Je referme cette année comme on referme un chapitre dense : avec de la fatigue heureuse, beaucoup d'apprentissages, et la conviction d'avoir participé à quelque chose qui me dépasse. La méthode OPTIQ avait pour ambition de révéler la place et la valeur de chacun dans une organisation. En contribuant à la rendre accessible, j'ai surtout découvert la mienne.

Annexes

Éléments de référence — non comptabilisés dans la pagination du mémoire : glossaire, cartographie des modules, pile technique et feuille de temps.

A1. Glossaire

Terme	Définition
Maquette	Version de démonstration d'une application, destinée à prouver un concept ou à convaincre, sans les garanties d'un logiciel exploité en clientèle.
Produit	Logiciel abouti, déployé, fiable et commercialisable, conçu pour être utilisé par plusieurs clients réels.
Savoirs	Connaissances théoriques mobilisées pour comprendre, analyser ou situer une activité : la base cognitive.
Savoir-faire	Capacités pratiques à mettre en œuvre des savoirs dans une situation concrète : l'action, la technique maîtrisée.
Aptitudes	Dispositions personnelles ou cognitives permettant d'agir efficacement (concentration, rigueur...).
HSC	Habiletés Socio-Cognitives. Compétences transversales liées au comportement, à l'autonomie, à la coopération (norme X50-766).
Rôle	Ensemble structuré d'activités confiées à une personne ; fonction transversale, indépendante du poste.
Activité	Unité de base du travail dans la méthode OPTIQ : un ensemble cohérent de tâches produisant un résultat concret.
Tâche	Action élémentaire réalisée dans le cadre d'une activité.
Entité	Dans DevOPTIQ, une organisation cliente avec ses propres données cloisonnées (activités, rôles, cartographie). Brique du multi-client.
Blueprint	Module Flask regroupant les routes d'un domaine fonctionnel. L'application en compte une cinquantaine.
FileBlob	Modèle stockant le contenu binaire des fichiers directement en base, pour survivre aux redémarrages du cloud.
Fallback	Comportement de repli déclenché quand un service (ex. l'IA) est indisponible, pour éviter toute panne visible.
SaaS	Software as a Service : logiciel hébergé et fourni en ligne, utilisé par abonnement plutôt qu'installé.
Cloud Run	Service Google d'exécution de conteneurs Docker, facturé à l'usage, au système de fichiers éphémère.
Commit / Push	Enregistrer localement un ensemble de modifications (commit) ; les transférer vers le dépôt distant (push).
API	Application Programming Interface : interface permettant à deux services d'échanger des données.

R.O.M.E 4.0	Répertoire Opérationnel des Métiers et des Emplois de France Travail, utilisé pour la projection métier.
-------------	--

A2. Cartographie des modules

Vue d'ensemble des grands domaines fonctionnels et de leurs modules (blueprints) — un aperçu de l'ampleur fonctionnelle évoquée en Partie II.

Domaine	Modules (extraits)
Cartographie	<code>cartography_editor</code> , <code>activities_map</code> , <code>vsd_x_*</code> (import Visio), synchronisation carto → base.
Activités	<code>activities</code> , <code>activities_view</code> , <code>activities_render</code> , <code>activities_data</code> , <code>constraints</code> , <code>tasks</code> , <code>tools</code> , <code>task_link_assignments</code> .
Compétences	<code>competences</code> , <code>competences_plan</code> , <code>savoirs</code> , <code>savoir_faires</code> , <code>aptitudes</code> , <code>softskills</code> , <code>skills</code> .
RH & comptes	<code>gestion_rh</code> , <code>gestion_compte</code> , <code>roles</code> , <code>roles_view</code> , <code>gestion_outils</code> .
Performance & temps	<code>performance</code> , <code>performance_personnalisee</code> , <code>time_view</code> , <code>time_extra</code> .
IA	<code>propose_*</code> (savoirs, savoir-faire, HSC, aptitudes), <code>propose_from_file</code> , <code>onboarding</code> , <code>chatbot</code> , <code>import_full</code> , <code>translate_softskills</code> , <code>projection_metier</code> .
Transverse	<code>connexion_routes</code> , <code>routes_password</code> , <code>settings</code> , <code>export</code> , <code>changelog</code> , <code>test_panel</code> .

A3. Pile technique

Couche	Technologie
Backend	Python 3, Flask, SQLAlchemy (Flask-SQLAlchemy)
Base de données	PostgreSQL (production) / SQLite (développement & tests)
Frontend	HTML Jinja2, CSS « vanilla » (charte rose/vert/blanc), JavaScript « vanilla » (sans framework)
Authentification	Sessions Flask, hachage des mots de passe (werkzeug)
Intelligence artificielle	API OpenAI (propositions, chatbot, onboarding, import assisté)
Cartographie	OptiqCarto — éditeur / viewer SVG maison, import VSDX
Tests	pytest — 979 tests, base SQLite temporaire, panneau <code>/testpanel</code>
Déploiement	Docker → Google Cloud Run, serveur Gunicorn
Outils	Git / dépôt distant, communication d'équipe (visio, comptes rendus)

A4. Feuille de temps

◆ À COMPLÉTER — FEUILLE DE TEMPS DE L'ALTERNANCE

Insérer ici la feuille de temps / le carnet de bord de l'année (comme dans le mémoire de 2^e année).
N'ayant pas ces données, je laisse l'emplacement prêt à recevoir le tableau ou le document scanné.



ANNEXES COMPLÉMENTAIRES ÉVENTUELLES

Captures d'écran de l'application (cartographie, fiche activité, synthèse des compétences), extraits de la documentation technique ou du guide utilisateur. À insérer selon les attendus de ton jury.

A5. Schéma de la base de données

La base de production (PostgreSQL, hébergée sur Neon) compte près de **29 tables**. Le schéma ci-dessous en donne une vue simplifiée, centrée sur les deux pivots de l'application : l'**entité** (la racine multi-locataire — tout en dépend par `entity_id`) et l'**activité** (le cœur métier issu de la cartographie, autour duquel gravitent tâches, compétences et analyses).

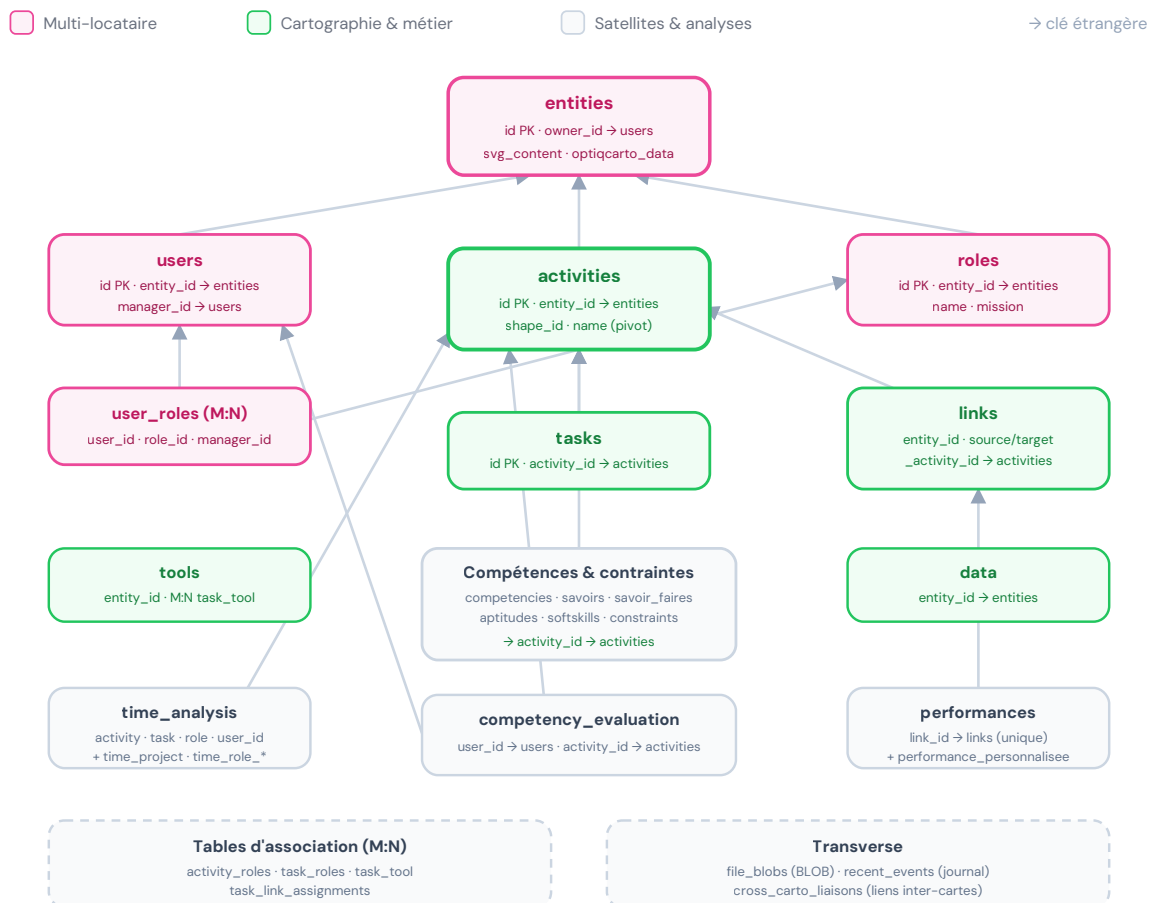


Figure 3. Schéma relationnel simplifié : `entités` (racine multi-locataire) et `activités` (pivot métier) structurent l'ensemble. « PK » = clé primaire.

Note : ce schéma est volontairement simplifié pour la lisibilité. Les tables de compétences (6) et d'analyse du temps (4) sont regroupées, et seules les relations principales sont tracées. Le détail exact des colonnes figure dans [Code/models/models.py](#).